HOTEL MASTER

Hotel Technology Showcase Website

PRODUCT REQUIREMENTS & ARCHITECTURE DOCUMENT

Version 1.0 · 2026 · Yohannes Bayeh

Project Name	Hotel Master (Hotel Technology Showcase Website)
Type	Full-Stack Hospitality Showcase & Contact Gateway API
Developer	Yohannes Bayeh
Release Year	2026
Repository Structure	Monorepo — /client (React 19 SPA) + /server (Node.js Express 5 API)
Primary UI Color	Luxury Gold — #D9A74A / #C9A84C
Stack	React 19 · Vite 8 · Tailwind CSS v4 · Lucide React · Node.js · Express 5 · Brevo v3 SMTP API
Frontend (Live)	https://hotel-master-qfcz78r5v-john-bayehs-projects.vercel.app/
Backend API (Live)	https://hotel-master-api.railway.app/
Deployment	Vercel (Frontend SPA) + Railway / Render (Backend API)

SECTION 1 — EXECUTIVE SUMMARY, PROBLEM STATEMENT & SOLUTION

1.1 Executive Summary

Hotel Master is a full-stack hospitality showcase platform that pairs a modern React 19 single-page application with a dedicated Node.js/Express 5 API layer. The system is purpose-built for boutique hotels seeking a polished digital storefront to present technology integrations, in-room amenities, and guest-facing digital services, while simultaneously providing a secure, automated channel for prospective clients and partners to submit consultation inquiries directly to hotel management.

Architecturally, the project is organized as a monorepo split between a /client SPA (built with Vite 8, styled with Tailwind CSS v4, and iconified with Lucide React) and a /server REST API responsible for serving structured content and processing contact submissions. The visual identity is a dark, luxury-hospitality aesthetic — Ebony backgrounds, Slate content cards, Georgia serif headings, and a Luxury Gold accent (#D9A74A / #C9A84C) — designed to communicate premium positioning to prospective hotel clients.

1.2 Problem Statement

Problem	Impact
Boutique hotels lack a dedicated digital showcase for their technology stack and amenities	Prospective guests and partners cannot evaluate a property's digital/tech offering before engaging
Consultation and partnership inquiries are handled manually via email or phone	Slow response times, lost leads, and inconsistent intake of prospective client information
No structured, secure inbound channel for contact submissions	Exposure to spam, abuse, and unvalidated/unsafe input reaching hotel staff inboxes
Static marketing sites cannot serve dynamic service or amenity content	Content updates require developer intervention rather than a simple API-driven update
No safeguards against high-volume or malicious form submissions	Risk of denial-of-service via submission flooding and cross-site scripting (XSS) payloads

1.3 Solution

- A React 19 single-page application that presents hotel services, amenities, and brand story through a luxury dark-themed interface, consuming data dynamically from a dedicated REST API.
- A Node.js + Express 5 backend exposing versioned, purpose-specific endpoints for services, brand content, health monitoring, and contact intake.
- A hardened POST /api/contact endpoint combining in-memory rate limiting, HTML sanitization, strict payload size limits, and CORS origin whitelisting to guarantee safe, resilient inbound lead capture.
- Transactional email delivery via the Brevo v3 SMTP API, invoked through resilient curl-based execution to ensure inquiry notifications reach hotel management reliably.

- A decoupled deployment model — Vercel for the frontend SPA and Railway/Render for the backend API — enabling independent scaling and release cycles for each tier.

SECTION 2 — CORE FEATURES & FUNCTIONAL REQUIREMENTS

2.1 Functional Requirements

ID	Requirement	Description
FR-01	Dynamic Services Showcase	Render hotel services and technology amenities sourced live from GET /api/services
FR-02	Brand & About Presentation	Display hotel brand story and technology highlights sourced from GET /api/about
FR-03	Consultation Contact Gateway	Allow visitors to submit consultation inquiries via a validated, rate-limited contact form
FR-04	Automated Email Notification	Deliver submitted inquiries to hotel management automatically via Brevo transactional email
FR-05	API Health Monitoring	Expose a lightweight healthcheck endpoint for uptime and deployment verification
FR-06	Abuse & Spam Prevention	Enforce per-IP rate limiting and payload size limits on all inbound contact submissions
FR-07	Cross-Origin Access Control	Restrict API access to explicitly whitelisted frontend origins via environment configuration
FR-08	Responsive Luxury UI	Present all content in a dark, high-contrast, serif-accented luxury design system across devices

2.2 Non-Functional Requirements

Category	Requirement
Security	Input sanitization (HTML escaping) on all contact form fields to prevent stored/reflected XSS
Security	Reverse proxy trust configuration (app.set('trust proxy', 1)) for accurate client IP resolution behind Railway/Render proxies
Reliability	Resilient email dispatch via curl-based execution against the Brevo v3 SMTP API
Performance	Lightweight JSON REST services layer with no database dependency for content delivery
Scalability	Independent deployment and scaling of frontend (Vercel) and backend (Railway/Render) tiers
Availability	Dedicated /api/health endpoint supporting external uptime monitoring

SECTION 3 — FULL-STACK ARCHITECTURE & MONOREPO STRUCTURE

3.1 Repository Topology

Hotel Master is organized as a single monorepo containing two independently deployable applications:

Directory	Responsibility	Key Technologies
/client	React 19 single-page application delivering the public-facing hotel showcase UI	React 19 · Vite 8 · Tailwind CSS v4 · Lucide React
/server	Node.js REST API serving content endpoints and processing contact submissions	Node.js · Express 5 · Brevo v3 SMTP API

3.2 Frontend Architecture (/client)

- React 19 powers the component tree and rendering lifecycle for the single-page application.
- Vite 8 serves as the build tool and development server, providing fast bundling and hot module replacement.
- Tailwind CSS v4 implements the utility-first styling system underpinning the luxury dark design language.
- Lucide React supplies the icon set used throughout the services, amenities, and navigation UI.
- The SPA consumes the backend REST API at runtime to populate services, about/brand content, and to submit contact inquiries.

3.3 Backend Architecture (/server)

- Node.js with Express 5 implements the REST API layer, routing, and middleware pipeline.
- Content (services and about/brand data) is served through a structured JSON REST API services layer, with no relational or NoSQL database dependency.
- Contact submissions are processed through a dedicated, hardened endpoint incorporating rate limiting, sanitization, and payload constraints (see Section 4).
- Outbound transactional email is dispatched to the Brevo v3 SMTP API using curl execution for resilient, dependency-light delivery.

3.4 Deployment Topology

Tier	Platform	Live Endpoint
Frontend (SPA)	Vercel	hotel-master-qfcz78r5v-john-bayehs-projects.vercel.app
Backend (API)	Railway / Render	hotel-master-api.railway.app

SECTION 4 — SECURITY ARCHITECTURE

4.1 Overview

The /api/contact endpoint is the primary attack surface of Hotel Master, as it accepts unauthenticated input from public visitors. Accordingly, it is protected by a layered set of controls spanning network-level trust configuration, request throttling, payload constraints, and input sanitization.

4.2 Security Controls

Control	Implementation Detail	Purpose
Rate Limiting	In-memory limiter — max 5 requests per IP per 15-minute window	Prevents submission flooding and brute-force abuse of the contact endpoint
Input Sanitization	HTML escaping / sanitization applied to all incoming contact fields	Mitigates stored and reflected Cross-Site Scripting (XSS) attacks
Payload Size Limiting	Request body capped at 10kb	Prevents oversized or malformed payloads from reaching the application layer
Reverse Proxy Trust	app.set('trust proxy', 1)	Ensures accurate client IP resolution when the API sits behind Railway/Render's proxy layer, so rate limiting keys on the true client IP
CORS Whitelisting	Strict origin filtering driven by the CLIENT_ORIGIN environment variable	Restricts API access to only the authorized frontend origin(s), blocking cross-origin abuse

4.3 Design Rationale

Because the services and about endpoints are read-only and unauthenticated by design, the security posture of Hotel Master is deliberately concentrated on the single write path — the contact endpoint. Rate limiting and CORS whitelisting operate at the request-admission layer, while sanitization and payload limits operate at the data-validation layer, providing defense in depth without introducing the operational overhead of a full authentication system for a public marketing surface.

SECTION 5 — API SPECIFICATION

5.1 Endpoint Summary

Method	Endpoint	Description
GET	/api/services	Returns dynamic hotel services & technology amenities data
GET	/api/about	Returns hotel brand story & technology highlights
GET	/api/health	API healthcheck / uptime status endpoint
POST	/api/contact	Accepts and processes consultation inquiry submissions

5.2 Endpoint Detail — GET /api/services

Attribute	Detail
Method / Path	GET /api/services
Auth	None (public, read-only)
Response	JSON array of hotel service & amenity objects
Typical Status Codes	200 OK · 500 Internal Server Error

5.3 Endpoint Detail — GET /api/about

Attribute	Detail
Method / Path	GET /api/about
Auth	None (public, read-only)
Response	JSON object containing hotel brand story and technology highlights
Typical Status Codes	200 OK · 500 Internal Server Error

5.4 Endpoint Detail — GET /api/health

Attribute	Detail
Method / Path	GET /api/health
Auth	None (public)
Response	JSON status object confirming API availability
Typical Status Codes	200 OK

5.5 Endpoint Detail — POST /api/contact

Attribute	Detail
Method / Path	POST /api/contact
Auth	None (public — protected via rate limiting & CORS whitelist)
Request Body Limit	10kb maximum payload size
Request Body (typical fields)	name, email, message / consultation details
Server-Side Processing	HTML escaping & sanitization, rate-limit check, CORS origin validation, then dispatch via Brevo v3 SMTP API
Rate Limit	Maximum 5 requests per IP per 15-minute window (in-memory)
Response (Success)	200 OK — JSON confirmation that the inquiry was received and dispatched
Response (Failure)	400 Bad Request (validation) · 429 Too Many Requests (rate limit exceeded) · 500 Internal Server Error (dispatch failure)

SECTION 6 — EMAIL INTEGRATION & DEPLOYMENT

6.1 Brevo Transactional Email Integration

Contact inquiries submitted through the /api/contact endpoint are relayed to hotel management via the Brevo v3 SMTP API (api.brevo.com/v3/smtp/email). Rather than relying on a Node.js SMTP client library, the integration executes outbound requests via curl, chosen for resilience and minimal dependency footprint in constrained or ephemeral hosting environments.

Attribute	Detail
Provider	Brevo (formerly Sendinblue)
API Version	v3 SMTP API
Endpoint	api.brevo.com/v3/smtp/email
Execution Method	curl-based HTTP request execution for resilient, dependency-light delivery
Trigger	Successful validation & sanitization of a POST /api/contact submission
Recipient	Hotel management (configured notification address)

6.2 Deployment Architecture

Component	Platform	Notes
Frontend (/client)	Vercel	Static SPA build deployment; live at hotel-master-qfcz78r5v-john-bayehs-projects.vercel.app
Backend (/server)	Railway / Render	Node.js/Express API deployment; live at hotel-master-api.railway.app

The frontend and backend are deployed and versioned independently. The SPA communicates with the API exclusively over HTTPS, with the API enforcing CORS restrictions via the CLIENT_ORIGIN environment variable so that only the authorized Vercel-hosted frontend origin may issue cross-origin requests.

6.3 Design System & Luxury UI Aesthetics

Design Token	Value	Usage
Primary Accent — Luxury Gold	#D9A74A / #C9A84C	Headings, interactive accents, call-to-action elements, section dividers
Background — Dark Ebony	#080808	Primary page background across the SPA
Surface — Slate Cards	#111111	Content cards, panels, and elevated surfaces
Typography — Headings	Georgia / Serif	High-contrast serif headings reinforcing a luxury hospitality identity

Design Token	Value	Usage
Iconography	Lucide React	Consistent line-icon system across services, amenities, and navigation

SECTION 7 — DOCUMENT CONTROL

Field	Detail
Document Title	Hotel Master — Product Requirements & Architecture Document
Version	1.0
Date	July 27, 2026
Author	Yohannes Bayeh
Status	Final — Based on Verified Codebase Details
Distribution	Internal / Project Reference